

FRC Junior Academy 2025 编程控制单元作业

作业概述

考虑到 FRC 相关的编程控制实践需要 WPILIB 环境和比较复杂的设备要求，因此本次作业将使用纯 Java 语言，考察学生对简单 Java 语法和面向对象编程的理解。

现代 FRC 比赛中有一个重要概念叫做“Coopertition”（合作竞争），这个词本身就是“Cooperation”（合作）和“Competition”（竞争）的结合。选手们常常需要在合作与竞争之间找到微妙的平衡，这本质上就是一种博弈论的思维。

举个例子：在 2025 年的赛题中，选手们可以选择和对手商量一起放置 Processor 来获得合作分数。这听起来很美好，对吧？但是这里有个“陷阱”：如果一方在比赛中突然改变主意，不遵守约定，那么遵守约定的队伍就会浪费宝贵的时间放置净得分 2 分的 Processor，而另一方则可以把时间放在全力得分上，从而获得更高的分数，尝试赢得比赛

这就是典型的囚徒困境：

- 如果双方都遵守约定（合作），大家都能获得不错的分数
- 如果一方背叛约定，背叛者可能获得更高分数从而赢得比赛，而遵守者会吃亏
- 如果双方都背叛，都得不到想要的 coopertition 分数

囚徒困境是博弈论里最著名的模型之一，完美地刻画了这种“两难”处境：个人理性选择与集体最优结果之间的冲突。

在这个作业中，你将设计一个智能策略来参与“三人囚徒困境”游戏。这是一个经典的博弈论问题，通过编程实现可以帮助你理解策略设计和决策逻辑。

什么是囚徒困境？

经典二人囚徒困境故事

让我们从一个经典的故事开始理解囚徒困境：

故事背景：两个小偷被警察抓获，分别关在不同的审讯室里。警察没有足够的证据定罪，但可以以较轻的罪名（比如非法持有工具）判处每人 1 年监禁。

警察的提议：

1. 如果两个人都保持沉默（合作），每人判 1 年
2. 如果一个人背叛同伙，背叛者无罪释放，被背叛者判 3 年
3. 如果两个人都背叛对方，每人判 2 年

困境所在：

1. 从个人角度：背叛总是比合作更有利（0 年 vs 1 年，或 2 年 vs 3 年）

2. 从集体角度：合作对双方都更有利（总共 2 年 vs 总共 4 年）
这就是”困境”：个人理性选择导致集体次优结果。

扩展到三人囚徒困境

在我们的编程作业中，我们将这个经典问题扩展到三人版本：

游戏设定：

- 三个玩家同时进行游戏
- 每轮每个玩家选择”合作”或”背叛”
- 根据所有玩家的选择，每个玩家获得相应的分数
- 游戏进行多轮，最终看谁的平均分数最高

游戏规则详解

基本规则

- 每轮游戏，三个玩家同时选择合作 (0) 或背叛 (1)
- 根据所有玩家的选择，每个玩家获得相应的分数
- 游戏进行约 100 轮
- 最终分数是每轮分数的平均值

收益矩阵（三人版本）

让我们看看当玩家 1 选择不同策略时的收益情况：

玩家 1 选择合作时的收益

玩家 2	玩家 3	玩家 1 得分	说明
合作	合作	6 分	最佳合作结果
合作	背叛	3 分	被一人背叛
背叛	合作	3 分	被一人背叛
背叛	背叛	0 分	被两人背叛

玩家 1 选择背叛时的收益

玩家 2	玩家 3	玩家 1 得分	说明
合作	合作	8 分	背叛两个合作者，最高分 与另一背叛者平分
合作	背叛	5 分	
背叛	合作	5 分	与另一背叛者平分
背叛	背叛	2 分	三人互相背叛

策略分析

从这个收益矩阵可以看出：

- **背叛两个合作者是最有利的（8 分）**
- **三人合作是次优选择（6 分）**
- **被背叛是最不利的（0 分）**

你的任务

设计一个 Java 类，实现你的游戏策略。你的类必须：

1. 继承自 Player 类
2. 实现 selectAction 方法
3. 根据游戏历史决定下一步行动

学习资源和使用 AI 工具

鼓励使用 AI 工具

我们鼓励你使用现代 AI 工具来帮助设计和优化你的策略，比如：

- **DeepSeek、Kimi、Qwen** 等大语言模型
- 可以询问 AI 关于博弈论策略的建议
- 可以请 AI 帮助分析代码逻辑
- 可以请 AI 提供策略优化建议

讨论和合作

- 你可以与同学讨论策略思路
- 可以分享和比较不同的策略方法
- 鼓励团队合作，但最终代码必须是你自己编写的

学习目标

通过这个作业，你将学会：

- 理解博弈论的基本概念
- 设计算法策略
- 使用 Java 面向对象编程
- 分析复杂问题并找到解决方案

使用模板快速开始

```
public class GongZeruiPlayer extends ThreePrisonersDilemma.Player {

    /**
     * 选择下一步行动
     * @param n 当前轮数（从 0 开始）
     * @param myHistory 我的历史行动数组
     * @param oppHistory1 对手 1 的历史行动数组
     * @param oppHistory2 对手 2 的历史行动数组
     * @return 0 表示合作，1 表示背叛
     */
    @Override
    int selectAction(int n, int[] myHistory, int[] oppHistory1, int[] oppHistory2) {
        // 在这里实现你的策略逻辑

        // 示例：总是合作（最简单的策略）
        return 0;
    }
}
```

策略示例和思路

1. 友好玩家 (NicePlayer)

```
// 总是选择合作
return 0;
```

思路：相信合作能带来长期利益，适合与友好玩家对战。

2. 恶劣玩家 (NastyPlayer)

```
// 总是选择背叛
return 1;
```

思路：追求短期最大利益，但可能被其他玩家针对。

3. 随机玩家 (RandomPlayer)

```
// 随机选择合作或背叛
if (Math.random() < 0.5)
    return 0; // 50% 概率合作
else
    return 1; // 50% 概率背叛
```

思路：不可预测的策略，避免被针对。

4. 以牙还牙 (T4TPlayer)

```
if (n == 0) return 0; // 第一轮合作
// 随机选择一个对手，模仿他上一轮的行动
if (Math.random() < 0.5)
    return oppHistory1[n-1];
else
    return oppHistory2[n-1];
```

思路：以合作开始，然后模仿对手的行为。

5. 统计策略 (StatisticalPlayer)

```
if (n < 5) return 0; // 前几轮合作，收集信息

// 统计对手的合作率
int coop1 = 0, coop2 = 0;
for (int i = 0; i < n; i++) {
    if (oppHistory1[i] == 0) coop1++;
    if (oppHistory2[i] == 0) coop2++;
}

double rate1 = (double) coop1 / n;
double rate2 = (double) coop2 / n;

// 如果对手合作率都很高，继续合作；否则背叛
if (rate1 > 0.7 && rate2 > 0.7) {
    return 0;
} else {
    return 1;
}
```

思路：根据对手的历史行为调整策略。

高级策略思路，一些可选的思路

1. 适应性策略

- 根据游戏进展动态调整策略
- 在早期合作，后期根据情况调整

2. 模式识别

- 寻找对手行为中的规律
- 预测对手的下一步行动

3. 风险评估

- 计算不同选择的风险和收益
- 选择风险收益比最优的策略

4. 心理策略

- 故意表现出某种模式，然后突然改变
- 利用对手的预期心理

测试你的策略

在提交作业之前，你可以使用 `ThreePrisonersDilemma.java` 文件来测试你的策略！

测试步骤：

1. 准备文件：

- 将你的策略文件(如 `GongZeruiPlayer.java`)放在与 `ThreePrisonersDilemma.java` 同一目录下

2. 修改测试文件：

- 打开 `ThreePrisonersDilemma.java`
- 找到文件开头的”学生测试区域”
- 将 `numPlayers = 6` 改为 `numPlayers = 7`
- 在 `switch` 语句中添加你的策略：

```
case 6: return new GongZeruiPlayer(); // 替换为你的类名
```

3. 编译和运行:

```
javac ThreePrisonersDilemma.java GongZeruiPlayer.java
java ThreePrisonersDilemma
```

4. 查看结果:

- 程序会显示你的策略与其他策略的对战结果
- 最后会显示所有策略的排名和分数

重要提示:

- ThreePrisonersDilemma.java 仅用于测试, 提交作业时不需要提交此文件
- 只需要提交你的策略文件和说明文件

提交要求

必需文件

1. **Java 文件:** 你的姓名拼音 Player.java
 - 文件名格式: GongZeruiPlayer.java (例如: 龚泽睿 → GongZeruiPlayer.java)
 - 只包含一个类, 类名为 GongZeruiPlayer (首字母大写, 无下划线)
 - 继承 ThreePrisonersDilemma.Player
 - 实现 selectAction 方法

命名规则:

- 文件名: 你的姓名拼音 Player.java, 比如 GongZeruiPlayer.java
- 类名: 首字母大写, 如 GongZeruiPlayer
- 注意: Java 不允许文件名包含中文字符, 所以必须使用拼音

必需文件

1. **Java 文件:** 你的姓名拼音 Player.java
 - 文件名格式: GongZeruiPlayer.java
2. **说明文件:** 你的中文姓名.txt
 - 文件名格式: 龚泽睿.txt
 - 可以包含你的策略设计思路 (可选)
 - 可以解释为什么选择这种策略 (可选)
 - 如果不想写说明, 可以提交空文件

评分标准

- **策略效果 (60%)**: 你的策略在锦标赛中的表现
- **代码质量 (25%)**: 代码的可读性、注释和结构
- **创新性 (15%)**: 策略的原创性和思考深度

提交方式

将你的 Java 文件发送给老师，老师会：

1. 将你的策略添加到测试系统中
2. 运行完整的锦标赛
3. 返回你的策略的最终排名和分数

提示和建议

设计策略时考虑的问题：

1. **理解游戏机制**: 仔细分析收益矩阵，理解不同选择的结果
2. **考虑长期利益**: 在重复游戏中，合作可能比短期背叛更有利
3. **观察对手行为**: 利用历史信息来预测对手的未来行动
4. **平衡风险收益**: 在追求高分和避免低分之间找到平衡
5. **适应不同对手**: 设计能应对各种类型对手的策略

编程建议：

1. **测试你的策略**: 可以手动模拟几轮游戏来验证你的逻辑
2. **添加注释**: 解释你的策略思路，让代码更容易理解
3. **模块化设计**: 将复杂策略分解为多个方法
4. **边界情况处理**: 考虑第一轮、最后一轮等特殊情况
5. **测试你的策略**: 可以手动模拟几轮游戏来验证你的逻辑

使用 AI 工具的建议：

1. **明确问题**: 向 AI 清楚地描述你想要解决的问题
2. **验证建议**: 不要盲目接受 AI 的建议，要理解其逻辑
3. **迭代改进**: 基于 AI 的建议进行改进和优化

4. 学习原理：理解 AI 建议背后的博弈论原理

学习资源

- 博弈论基础
- 囚徒困境详解
- Java 面向对象编程教程

祝你好运！设计出最聪明的策略吧！记住，最好的策略往往不是最复杂的，而是最适合的。